

Комната №4 — Группа №4

Болорболд Аригуун, Р3411

Агей Михаил, Р3417

ПРАКТИКУМ №1 СТРОИМ КАРТУ НАШЕГО "ОБЛАЧНОГО ФАЙЛОВОГО ХРАНИЛИЩА"

- Шаг 1: Определяем нашу систему
- Шаг 2: Определяем акторов - Кто взаимодействует с системой?
- Шаг 3: Определяем внешние системы - С чем мы интегрируемся?
- Шаг 4: Определяем взаимодействия - Что передается через границы доверия?
- Подводим итог

Шаг 1:

Облачное файловое хранилище — это распределенная система для хранения, управления и совместного использования файлов через интернет с доступом из любого устройства.

Ключевые компоненты системы:

- Серверная часть
 - API-шлюзы
 - Микросервисы для обработки файлов, метаданных, пользователей
- Хранилище данных
 - Объектное хранилище: (например, Amazon S3, Google Cloud Storage) для самих файлов
 - База данных: (например, PostgreSQL, MongoDB) для метаданных (название файла, размер, владелец, путь, права доступа)
 - Кеш: (например, Redis) для сессий и временных данных для ускорения работы
- Клиентская часть (фронтэнд)

- Инфраструктура
 - Виртуальные машины/Контейнеры: (Docker, Kubernetes) для развертывания сервисов
 - Балансировщики нагрузки: Распределяют трафик между серверами
- Сервисы
 - Сервис аутентификации и авторизации.
 - Сервис поиска и индексации.
 - Сервис фоновых задач: конвертация видео, автоочистка по TTL

Шаг 2:

Основные акторы:

1. Конечные пользователи
 - a. Физические лица
 - b. Бизнес-пользователи
 - c. Гости
2. Администраторы
 - a. Сисадмины
 - b. Админы безопасности
 - c. Админы техподдержки
3. Технические акторы
 - a. Разработчики
 - b. Автоматизированные лица (периодические скрипты и боты)
 - c. CI/CD
4. Системные акторы
 - a. Мониторинговые системы
 - b. Системы бэкапа и восстановления
 - c. Системы аналитики и отчётности

Шаг 3:

1. Системы аутентификации
 - a. OAuth2 провайдеры (Google, Microsoft, Yandex)
 - b. SSO (LDAP)
 - c. 2FA (Google authenticator)
2. Платежные системы
3. Инфраструктурные сервисы
4. Безопасность
 - a. Антивирусные сканеры

- b. DDoS-защита (Cloudflare)
 - c. Системы обнаружения угроз (SIEM)
- 5. Интеграции с приложениями
 - a. Office 365
 - b. Системы управления проектами (Jira, Youtrack)
 - c. CRM (Salesforce)
 - d. Мессенджеры (Телега)
- 6. Мониторинг и аналитика
 - a. Логирование (Slf4j2)
 - b. Метрики (Prometheus или Grafana)
 - c. Аналитика использования (Google Analytics)

Шаг 4:

Актор	Внешняя система	Что передается?
Конечный пользователь	Система OAuth2 (Google)	Запрос на авторизацию, перенаправление с кодом доступа.
Клиентское приложение	Наше облачное API	Запросы на загрузку/скачивание файлов, метаданные, токены доступа.
Наш backend	Объектное хранилище (AWS S3)	Файлы для сохранения, запросы на получение файлов.
Наш backend	База данных	Метаданные файлов, информация о пользователях, права доступа.
Наш backend	Сервис Email (SendGrid)	Запрос на отправку письма (подтверждение регистрации, уведомление о совместном доступе, сброс пароля).

Наш backend	Платежный шлюз (Stripe)	Данные подписки, ID пользователя для создания "клиента", запросы на списание средств.
Наш backend	CDN (Cloudflare)	Инвалидация кеша (если файл обновился), статистика скачиваний.
Системный администратор	Сервис мониторинга (Datadog)	Метрики (загрузка CPU, использование диска, latency), логи ошибок.
Сервис проверки на вирусы	Внешний антивирусный API	Хэш файла или сам файл (в изолированном окружении) для проверки.

ПРАКТИКУМ №2 'ИЩЕМ УЯЗВИМОСТИ' (АНАЛИЗ УГРОЗ)

Для КОНТЕКСТА (внешние взаимодействия):

- 'Как пользователь доказывает, что это именно он? Достаточно ли только пароля?'
- 'Как мы защищаем данные, когда они передаются между системой и мобильным приложением?'
- 'Что произойдет, если злоумышленник перехватит ключ API к нашему Email-сервису?'

Для КОНТЕЙНЕРОВ (внутренние компоненты):

- 'Где хранятся файлы пользователей? Кто имеет к ним доступ?'
- 'Как общаются между собой веб-сервер и база данных? Зашифрован ли этот канал?'
- 'Где хранятся пароли пользователей? Как они защищены?'

Как пользователь доказывает, что это именно он? Достаточно ли только пароля?

- Угрозы: Кража паролей, фишинг, брутфорс
- Уязвимости:
 - Слабые пароли
 - Отсутствие многофакторной аутентификации
- Защита:
 - Обязательная 2FA (SMS, приложения-аутентификаторы)
 - Политика сложных паролей
 - Блокировка после множества неудачных попыток
 -

Как мы защищаем данные, когда они передаются между системой и мобильным приложением?

- Угрозы: MITM-атаки, перехват трафика
- Уязвимости: Отсутствие шифрования трафика
- Защита:
 - Обязательное использование HTTPS/TLS
 - Certificate Pinning в мобильных приложениях
 - Проверка SSL-сертификатов

Что произойдет, если злоумышленник перехватит ключ API к нашему Email-сервису?

- Угрозы: Рассылка спама, фишинг, финансовые потери
- Уязвимости: Ключи в открытом доступе
- Защита:
 - Хранение в защищенных vault (HashiCorp Vault, AWS Secrets Manager)
 - Регулярная ротация ключей
 - Ограничение прав доступа для ключей

Где хранятся файлы пользователей? Кто имеет к ним доступ?

- Угрозы: Неавторизованный доступ, утечки данных
- Уязвимости: Публичные бакеты в S3, слабые ACL
- Защита:
 - Шифрование данных на rest (AES-256)
 - Строгие Bucket Policies
 - Логирование доступа через CloudTrail

Как общаются между собой веб-сервер и база данных? Зашифрован ли этот канал?

- Угрозы: Перехват внутреннего трафика
- Уязвимости: Незашифрованные соединения к БД
- Защита:
 - TLS/SSL для всех подключений к БД
 - Изоляция БД в private subnet
 - Strict firewall rules

Где хранятся пароли пользователей? Как они защищены?

- Угрозы: Утечка хэшей паролей
- Уязвимости: Слабые алгоритмы хэширования
- Защита:
 - Хэширование с salt (bcrypt, scrypt, Argon2)
 - Никогда не хранить в plain text
 - Регулярный аудит безопасности

ПРАКТИКУМ №3 МОДЕРНИЗИРУЕМ НАШЕ "ФАЙЛОВОЕ ХРАНИЛИЩЕ"

- Задача: "Используя изученные паттерны, предложите архитектурные изменения для устранения уязвимостей".

1. Паттерн: API Gateway + Authentication Service

- Проблема: Слабая аутентификация
- Решение:
 - Выделить аутентификацию в отдельный микросервис
 - API Gateway проверяет все входящие запросы
 - Единая точка для применения 2FA и политик паролей

2. Паттерн: Service Mesh (Istio/Linkerd)

- Проблема: Незашифрованная коммуникация между сервисами
- Решение:
 - Automatic mTLS для всех внутренних соединений
 - Certificate rotation и управление ключами
 - Единый контроль сетевой политики

3. Паттерн: Secret Management Vault

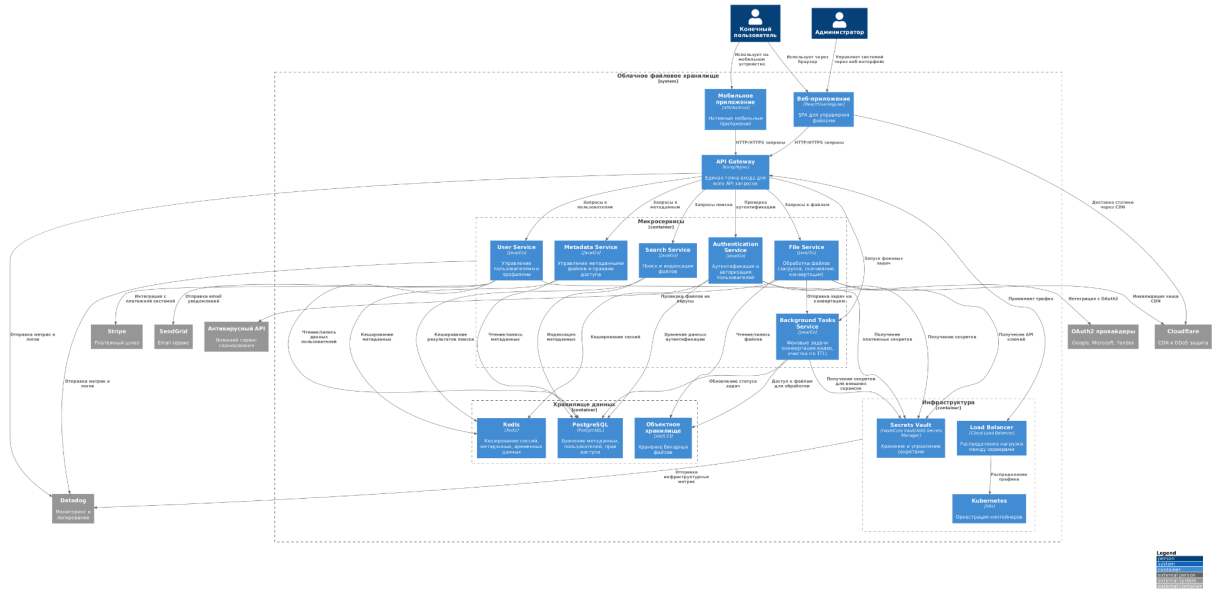
- Проблема: Утечка API-ключей и секретов
- Решение:
 - HashiCorp Vault для хранения всех секретов
 - Dynamic secrets для БД и внешних сервисов
 - Audit trail доступа к секретам

4. Паттерн: Zero Trust Network Architecture

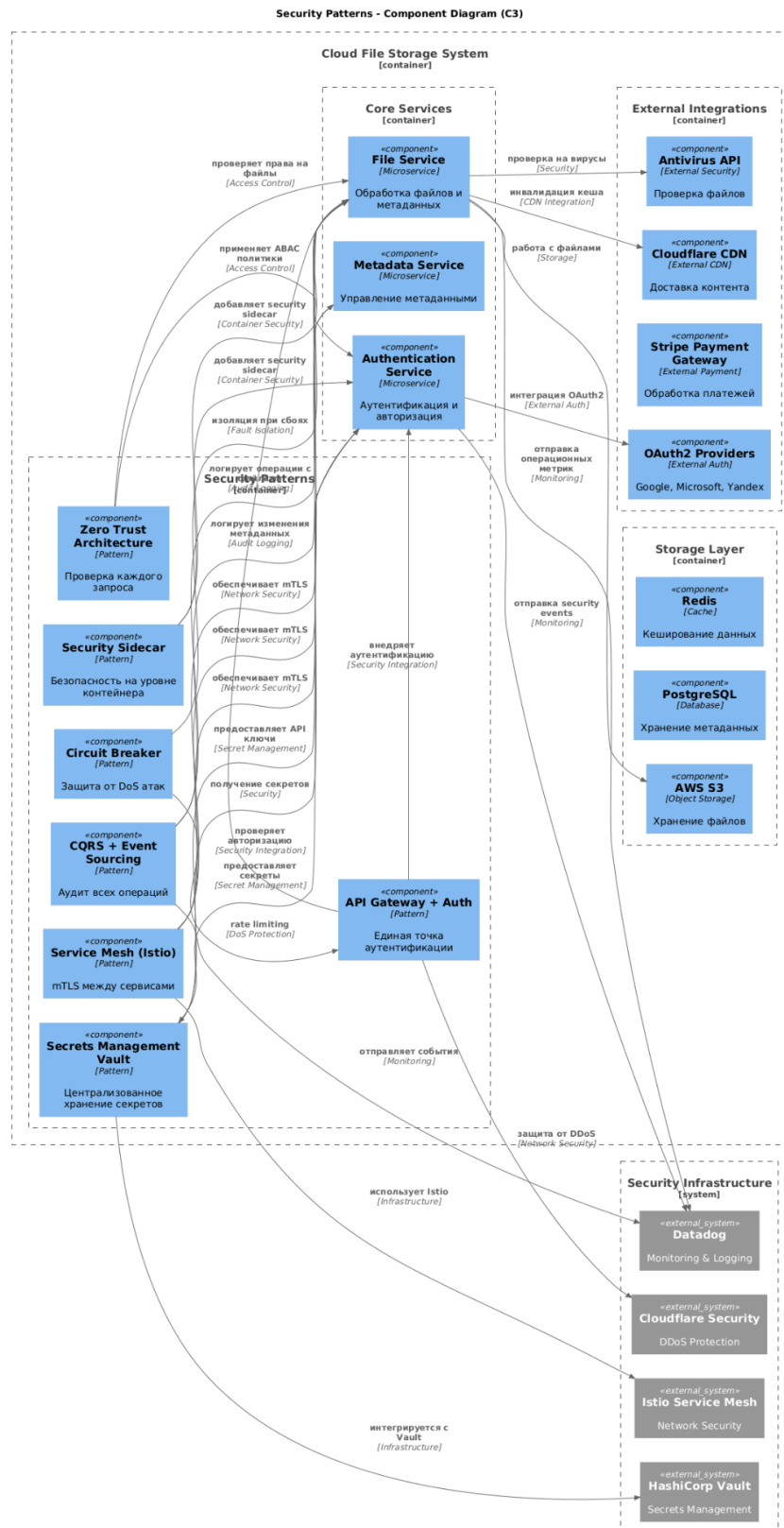
- Проблема: Неавторизованный доступ к файлам
- Решение:
 - Микросервис авторизации для каждого запроса к файлам

C2:

Общие функции системы - Container Diagram (C2)



C3:



C4:

